CUDA Tensor Compression with a Video Codec

15-618 Project Proposal | November 9, 2026
Team: Wayne Zhang and Alex Lin
Project website: https://github.com/a68lin/15618-project

Summary. This project explores if CUDA can speed up the process of compressing neural network weights with video codec (original weight compression algorithm adapted from LLM.265). We plan to implement a simple compression pipeline using the H.265 encoder, focusing on CUDA kernels that prepare tensors for encoding. We'll compare a CPU baseline with straightforward and optimized GPU implementations, and measure whether faster preprocessing actually improves end-to-end latency.

Background. A neural network weight matrix is a two-dimensional array of numbers. LLM.265 discoverd that we can arrange them into video frames and pass those frames to a video codec to compress the weight. Decoding and reversing the mapping then gives an approximation of the original matrix. Our project will reproduce a small version of this pipeline.

Video codecs are already heavily optimized, which isn't interesting. The interesting part is preparing neural network weights for these codecs, which aren't optimized. We want to see whether CUDA can make this preparation cheaper.

We will build this simple pipeline including computing scaling values, quantizing weights, packing frames, encoding, and decoding. Once scales are computed, individual values can be converted in parallel. For encoding and decoding, we will simply use an existing algorithm.

The challenge. There are several things we need to figure out for the CUDA implementation. Computing the scale for each row requires a reduction, and we need to find a good way to parallelize it. Another problem is packing the quantized weights into frames. The tensor layout and the buffer format expected by encoder can be different, so we may need to rearrange data.

These operations don't involve much computation, so memory access could be the main bottleneck. We want to try combining quantization and frame packing into one kernel, and experiment with different block sizes and memory layouts. But it's also possible that encoding and CPU/GPU transfers takes most of the time, so even a faster CUDA implementation won't improve total latency by much. We will have to measure this first.

Resources. We will write our CPU baseline and CUDA implementation in C++/CUDA. We will use LLM.265 and its repository [1, 2] as references, and may reuse their codec interface or NVIDIA Video Codec SDK samples [3]. We are not planning to implement the video codec ourselves.

Our main machine is a personal laptop with an NVIDIA RTX 4060 Laptop GPU (sm_89) and Intel Core i7-13650HX CPU. We also have access to catalyst-fleet, which has an AMD Ryzen Threadripper 3970X, RTX 5090 with 32GB memory, and RTX 2070 with 8GB memory. GHC and PSC Bridges-2 are other possible resources, although we haven't checked what hardware is available there. We will start with the laptop and use other machines if necessary.

For testing, we will use synthetic matrices first, then try some actual weights from a small open-source model. We haven't decided which model to use yet. CUDA events and a GPU profiler will be used to measure performance.

Goals and deliverables. Our main goal is to build a working compression and decompression pipeline, and compare the CPU baseline with a straightforward and an optimized CUDA implementation. We expect the GPU optimization to mainly involve parallel reductions, memory access, and combining quantization with frame packing. We will also check correctness of preprocessing and measure the error introduced by compression.

We will test different tensor sizes, for example square matrices from 1024 to 4096 elements per side, as well as some real model weight shapes. We want to measure the preprocessing time, total compression/decompression time, compression ratio, and reconstruction error. We will try to separate time spent on CUDA kernels, data transfers, and video encoding/decoding, so we can see where the time goes. For fair comparison, the different implementations will use the same compression settings. We will also include a simple quantization-only baseline to see whether using video codec actually helps.

After the basic optimization, we may try overlapping the preparation of one chunk with encoding of another using CUDA streams. Another possible experiment is changing how weights are arranged into frames. These are optional and depend on how much time we have. We don't have a specific speedup target yet since we haven't measured the baseline.

The final report will include performance comparisons, speedup plots, a breakdown of where time is spent, and the tradeoff between compression ratio and reconstruction error. We will focus on tensor-level error rather than testing full model accuracy.

If we run out of time, we will focus on the reduction and packing kernels, with fewer tensor sizes and one frame layout. If hardware H.265 encoding is difficult to set up, we can use a CPU encoder instead. It would make the end-to-end performance different, but we can still test our CUDA preprocessing optimizations.

Platform choice. We chose CUDA because the tensor processing involves doing similar operations on many values, which should benefit from GPU parallelism. CUDA also allows us to experiment with thread blocks, reductions, and memory access. We'll use C++ for the CPU baseline and for connecting to the video encoder. This also avoids including Python overhead in our main measurements.

Schedule.

November 9-15: Set up CUDA and H.265 encoding on the laptop. Choose some test matrices and implement the CPU baseline. We want to have a working compression/decompression pipeline by the end of this week.

November 16-22: Implement the first CUDA version, including reductions, quantization and packing. Check correctness and run some initial benchmarks. Prepare the milestone report due November 23.

November 23-29: Submit the milestone and start optimizing CUDA kernels. We will try combining quantization with packing, and experiment with block sizes and memory access. We will focus on these basic optimizations first because there are other coursework during this week.

November 30-December 6: Run more experiments with different tensor sizes and some actual model weights. Measure compression ratio and reconstruction error. If the core implementation works well, we may try overlapping computation and encoding. Start preparing the final results and plots.

December 7-11: Finish the final report, due December 10 at 11:59 PM, and update the project website. Prepare the poster for the 15-618 session on December 11, 8:30-11:30 AM. We may adjust the schedule depending on our progress.

References

[1] Ceyu Xu et al. LLM.265: Video Codecs are Secretly Tensor Codecs. MICRO 2025. https://doi.org/10.1145/3725843.3756078

[2] LLM.265 implementation: https://github.com/Entropy-xcy/llm.265

[3] NVIDIA Video Codec SDK: https://developer.nvidia.com/video-codec-sdk